LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [7] – [Abstract & Interface]



Nanda Mayla Adiasti

1031323400003

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Kelas Abstract (Abstract Class)

   Pengertian: Kata kunci abstrak (abstract) digunakan untuk mendeklarasikan method atau class yang belum memiliki implementasi lengkap. Kelas abstrak tidak dapat diinstansiasi menjadi sebuah Object secara langsung di dalam program, melainkan harus diturunkan (extended) ke kelas lain.

   Karakteristik:

   a. Hanya subclass (turunan) dari kelas abstrak yang dapat dijadikan Object setelah mengimplementasikan abstract method induknya.
   b. Direpresentasikan dalam diagram UML dengan nama kelas yang ditulis miring.
   c. Dapat berisi variabel biasa, konstruktor, method konkrit (memiliki implementasi), serta abstract method (tanpa implementasi).

2. Interface

   Pengertian: Interface adalah struktur yang mirip dengan kelas tetapi mendefinisikan seperangkat method atau konstanta yang akan diimplementasikan oleh objek lain. Interface berfungsi untuk mendefinisikan karakteristik atau kontrak perilaku suatu objek.

   Batasan dan Aturan pada Interface (Java 8):

   a. Semua atribut/variabel bersifat public, static, dan final (berfungsi sebagai konstanta).
   b. Hanya memiliki default method dan static method (serta abstract method).
   c. Tidak boleh memiliki deklarasi konstruktor.

   Multiple Inheritance: Berbeda dengan kelas biasa yang hanya mendukung single inheritance (satu kelas induk), sebuah interface mengizinkan multiple inheritance, di mana sebuah kelas dapat mengimplementasikan lebih dari satu interface sekaligus.
   Realization: Hubungan antara kelas dan interface dalam UML dinamakan realization (realisasi), di mana kelas merealisasikan kontrak method yang ada pada interface.

3. Fitur Tambahan pada Interface (Java 8)

   Default Method: Memungkinkan pengembang menambahkan method baru ke dalam interface dengan menggunakan kata kunci default beserta blok isinya, sehingga kelas yang mengimplementasikannya tidak wajib menulis ulang method tersebut.

   Static Method: Serupa dengan default method, static method di dalam interface memiliki implementasi langsung dan dapat dipanggil langsung melalui nama interface-nya tanpa perlu di-override oleh kelas implementasinya.

4. Perbedaan Utama Abstract Class dan Interface

Perbedaan	Abstract Class	Interface
Implementasi Method	Dapat memiliki method konkrit (dengan body) maupun abstract method.	Semua method secara default abstrak (tidak ber-body), kecuali default dan static method pada Java 8.
Atribut / Variabel	Dapat mendeklarasikan berbagai jenis variabel layaknya kelas biasa.	Hanya dapat mendefinisikan konstanta (public static final).
Pewarisan (Inheritance)	Hanya mendukung single inheritance (hanya bisa punya satu kelas induk).	Mendukung multiple inheritance (sebuah kelas/interface dapat mewarisi banyak interface).
Konstruktor	Memiliki konstruktor untuk inisialisasi oleh subclass.	Tidak memiliki konstruktor sama sekali.

B. Guided

1. Guided 1

[Employee.java]

```
package guided1;

// Fondasi umum: belum punya peran spesifik, jadi tidak boleh diinstansiasi langsung
public abstract class Employee {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // Aturan kosong: setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();
}
```

[SalariedEmployee.java]

```
package guided1;

// wajib menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

[Main.java]

```
package guided1;

public class Main {
    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("Mayla", 5000000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" + salaried.earnings());
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul07 ---
Nama: Mayla
Gaji: Rp5000000.0
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.226 s
Finished at: 2026-11-02T11:21:20+07:00
------------------------------------------------------------------------
```

2. Guided 2

[Payable.java]

```
package guided2;

public interface Payable {
    double getPaymentAmount();
}
```

[Employee.java]

```
package guided2;

// Fondasi umum: belum punya peran spesifik, jadi tidak boleh diinstansiasi langsung
public abstract class Employee implements Payable {

    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // Method abstract: setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

[SalariedEmployee.java]

```
package guided2;

//wajib menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {

    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
```

```
    public double earnings() {
        return weeklySalary;
    }
}
```

[Invoicee.java]

```
package guided2;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }
}
```

[Main.java]

```
package guided2;

public class Main {
    public static void main(String[]args){
        Payable[] items = new Payable[2];
        items[0] = new Invoice("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Mayla", 5000000);

        //Dynamic dispatch: Java otomatis mengeksekusi rumus yang tepat
        for (Payable item : items){
            System.out.println("Jumlah yang perlu dibayar: Rp" + item.getPaymentAmount());
        }
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul07 ---
Jumlah yang perlu dibayar: Rp3000000.0
Jumlah yang perlu dibayar: Rp5000000.0
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.058 s
Finished at: 2026-11-02T11:26:02+07:00
------------------------------------------------------------------------
```

3. Guided 3

[Payable.java]

```
package guided3;

public interface Payable {
    double getPaymentAmount();

    default void printReceipt(){
        System.out.println("Struk Resmi - Total: Rp" + getPaymentAmount());
    }

    static void logTransaction(String message){
        System.out.println("[AUDIT LOG]" + message);
    }

}
```

[Employee.java]

```
package guided3;


// Fondasi umum: belum punya peran spesifik, jadi tidak boleh diinstansiasi langsung
public abstract class Employee implements Payable {

    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // Method abstract: setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

[SalariedEmployee.java]

```
package guided3;

//wajibb menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {

    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

[Invoice.java]

```
package guided3;

public class Invoice implements Payable{
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }
}
```

[Main.java]

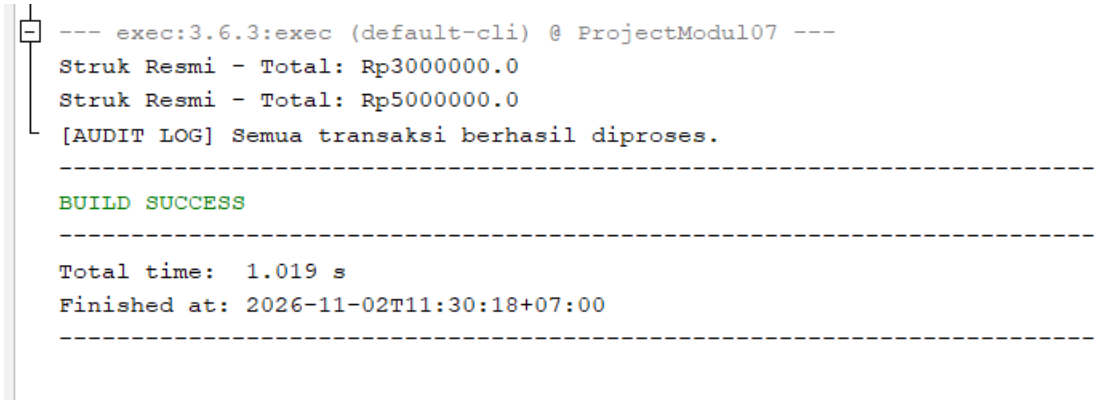
```
package guided3;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];
        items[0] = new Invoice("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Mayla", 5000000);

        for (Payable item : items) {
            item.printReceipt();
        }

        Payable.logTransaction(" Semua transaksi berhasil diproses.");
    }
}
```

Output :



4. Unguided

   Soal Unguided

   Soal — Sistem Kasir Terpadu Bioskop (*Cinema POS*)

   Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli **Tiket Nonton**, tetapi juga bisa membeli makanan/minuman (**Paket Snack F&B**).

   Secara konsep sistem, **Tiket Bioskop** dan **Paket Snack** berasal dari silsilah keluarga (hierarki) yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak (tidak bisa dibeli tanpa jenis studio). Namun, di mata program kasir, tiket dan makanan harus bisa disatukan, dihitung total tagihannya, dan dicetak di dalam satu struk secara otomatis.

   Pengerjaan soal ini dibagi menjadi 4 tugas utama untuk melatih abstraksi kelas, antarmuka (*interface*), dan fitur modern Java 8+.

TiketBioskop.java

```
package unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    // Abstract method khusus untuk perhitungan internal tiket
    public abstract double hitungTotalBayar();

    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

TiketReguler.java

```
package unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int
nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
```

```
        }

        @Override
        public String getNamaTransaksi() {
            return "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm() + "
    (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
        }
}
```

TiketVIP.java

```
    package unguided;

    public class TiketVIP extends TiketBioskop {
        private double biayaFasilitasVIP;

        public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double
    biayaFasilitasVIP) {
            super(judulFilm, hargaDasar, jumlahKursi);
            this.biayaFasilitasVIP = biayaFasilitasVIP;
        }

        @Override
        public double hitungTotalBayar() {
            return (getHargaDasar() + biayaFasilitasVIP) * getJumlahKursi();
        }

        @Override
        public String getNamaTransaksi() {
            double totalHargaPerKursi = getHargaDasar() + biayaFasilitasVIP;
            return "Tiket VIP Premiere - " + getJudulFilm() + " (" + getJumlahKursi() + "
    x Rp" + totalHargaPerKursi + ")";
        }
}
```

PaketSnack.java

```
    package unguided;

    public class PaketSnack implements ItemTransaksi {
        private String namaPaket;
        private int jumlahPorsi;
        private double hargaPerPorsi;

        public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi) {
            this.namaPaket = namaPaket;
            this.jumlahPorsi = jumlahPorsi;
            this.hargaPerPorsi = hargaPerPorsi;
        }

        @Override
```

```
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" +
hargaPerPorsi + ")";
    }
}
```

Main.java

```
package unguided;

public class Main {
    public static void main(String[] args) {
        // 1. Uji Proteksi Abstraksi (Dijadikan komentar karena kelas abstrak tidak
bisa di-instansiasi langsung)
        // TiketBioskop test = new TiketBioskop("Test", 10000, 1);

        // 2. Buka Kasir
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan
#042.");

        // 3. Buat Objek Transaksi
        TiketReguler tiketReg = new TiketReguler("Interstellar", 40000, 2, 3);
        TiketVIP tiketVip = new TiketVIP("Dune: Part Two", 60000, 2, 25000);
        PaketSnack snack = new PaketSnack("Combo Popcorn Caramel + 2 Soda",
1, 55000);

        // 4. Array Polimorfik Heterogen
        ItemTransaksi[] items = new ItemTransaksi[3];
        items[0] = tiketReg;
        items[1] = tiketVip;
        items[2] = snack;

        // 5. Pemrosesan Dinamis (Loop) & Penghitungan Akumulatif
        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0;

        for (ItemTransaksi item : items) {
            item.cetakRincianItem(); // Memanggil default method interface
            grandTotal += item.getNominalBayar();
        }

        // 6. Tutup Kasir & Cetak Grand Total
        System.out.println("-----------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);
```

```
        System.out.println("==========================================
        ==================");
            ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
        }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul07 ---
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.121 s
Finished at: 2026-11-02T11:42:03+07:00
------------------------------------------------------------------------
```

Penjelasan :

a. ItemTransaksi.java (Interface)

Berfungsi sebagai kontrak utama (interface) yang menyatukan dua entitas berbeda (tiket bioskop dan paket snack) agar dapat diproses secara seragam dalam satu sistem kasir.

Memuat dua method abstrak (getNominalBayar() dan getNamaTransaksi()) yang wajib diimplementasikan oleh kelas yang menggunakannya.

Menerapkan fitur Java 8+, yaitu default method (cetakRincianItem()) untuk mencetak format baris struk secara dinamis dan static method (catatLogKasir()) untuk mencatat log aktivitas sistem kasir.

b. TiketBioskop.java (Abstract Class)

Merupakan kelas induk abstrak yang merepresentasikan konsep umum tiket bioskop dan mengimplementasikan interface ItemTransaksi.

Kelas ini tidak dapat diinstansiasi secara langsung (abstract class) guna melindungi integritas data dasar tiket (judul film, harga dasar, dan jumlah kursi).

Menyediakan implementasi awal untuk method getNominalBayar() yang memanggil abstract method hitungTotalBayar().

c. TiketReguler.java & TiketVIP.java (Concrete Subclass)

Merupakan kelas turunan konkrit dari TiketBioskop yang merepresentasikan jenis tiket reguler dan VIP.

Menggunakan kata kunci @Override untuk mewujudkan polimorfisme melalui penulisan ulang (override) method hitungTotalBayar() dan getNamaTransaksi() sesuai dengan rumus harga serta fasilitas masing-masing jenis tiket.

d. PaketSnack.java (Concrete Class - Non Tiket)

Merupakan kelas mandiri yang merepresentasikan produk makanan/minuman (F&B) dan mengimplementasikan langsung interface ItemTransaksi tanpa menjadi turunan dari TiketBioskop.

Membuktikan konsep fleksibilitas interface di mana objek non-tiket dapat disetarakan dengan tiket di dalam satu transaksi kasir.

e. Main.java (Kelas Eksekusi Utama)

Berfungsi sebagai titik masuk program (entry point) untuk menguji seluruh implementasi OOP.

Mendemonstrasikan pencegahan instansiasi kelas abstrak, pemanggilan method statis pada interface, penerapan array polimorfik heterogen (ItemTransaksi[]), perulangan dinamis (looping), serta penghitungan akumulatif grand total transaksi kasir.

5. Kesimpulan

Berdasarkan praktikum Modul 7 – Abstract & Interface yang telah dilakukan, dapat disimpulkan bahwa:

Penerapan Kelas Abstrak (Abstract Class): Kelas abstrak berhasil digunakan sebagai kerangka dasar (seperti Employee dan TiketBioskop) yang tidak dapat diinstansiasi secara langsung untuk melindungi integritas data, namun mewajibkan kelas turunannya (subclass) untuk mengimplementasikan abstract method yang dideklarasikan.

Penggunaan Interface dan Kontrak Perilaku: Interface (seperti Payable dan ItemTransaksi) berhasil diterapkan untuk mendefinisikan sekumpulan kontrak method tanpa implementasi fisik, yang kemudian direalisasikan (realization) oleh berbagai kelas konkrit yang memiliki hierarki berbeda.

Fitur Modern Java 8+: Pemanfaatan default method dan static method di dalam interface memungkinkan penambahan fungsionalitas baru (seperti pencetakan struk dan pencatatan log kasir) secara fleksibel tanpa merusak kode pada kelas-kelas yang mengimplementasikannya.

Polimorfisme Heterogen: Konsep polimorfisme terbukti berhasil menyatukan objek dari silsilah yang berbeda—seperti objek tiket bioskop dan paket makanan (F&B snack)—ke dalam satu struktur array tunggal (ItemTransaksi[]). Hal ini memungkinkan

pemrosesan data, penghitungan total tagihan secara akumulatif, dan pencetakan rincian transaksi dilakukan secara seragam dan dinamis dalam sistem kasir bioskop (Cinema POS).